AI Library Seat Occupancy Tracker

Complete Project Documentation & Viva Preparation Guide (React + Vite Frontend)

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Technology Stack
5. System Architecture
6. Detailed Data Flow
7. Backend — Deep Dive
8. Frontend — Deep Dive
9. AI / Machine Learning Component
10. Communication Protocols
11. Key Algorithms & Techniques
12. File-by-File Explanation
13. How to Run the Project
14. Screenshots / UI Walkthrough
15. Advantages & Limitations
16. Future Enhancements
17. Viva Questions & Answers (50+)
18. Glossary of Technical Terms

1. Project Overview

Project Title: AI Library Seat Occupancy Tracker

In Simple Terms: This project uses a camera (webcam) and Artificial Intelligence to automatically detect how many seats in a library (or any room) are occupied and how many are vacant — in real-time. It shows a live dashboard on a web browser where anyone can see which seats are free without physically going to the library.

How it works (the "elevator pitch"):

> A camera watches the library. An AI model (YOLOv8) runs on each video frame to detect people and their body outlines. The system checks which people are sitting inside pre-defined "seat zones" and marks those seats as occupied. This information is instantly pushed to a web dashboard over WebSocket, where users see a live video feed with colored seat overlays, an occupancy ring chart, and per-seat status chips — all updating in real-time.

---

2. Problem Statement

In college and public libraries:

- Students waste time walking to the library only to find no seats available.
- Library staff cannot easily track seat utilization for planning.
- Manual seat counting is inefficient, inaccurate, and not real-time.

Solution: An automated, AI-powered, real-time seat occupancy monitoring system that requires only a standard webcam and a computer.

---

3. Objectives

#	Objective
1	Detect people in a camera feed using computer vision (YOLOv8).
2	Map detected people to pre-defined seat zones to determine occupancy.
3	Stream live video with occupancy overlays to a web dashboard.
4	Provide real-time occupancy data via WebSocket for instant updates.
5	Allow administrators to visually calibrate seat zones via a drag-and-draw interface.
6	Minimize false positives through head-keypoint verification, temporal smoothing, and contour merging.

---

4. Technology Stack

4.1 Backend (Server + AI)

Technology	Purpose	Version
Python	Main programming language	3.10+
FastAPI	Web server framework (handles HTTP + WebSocket)	0.115.0
Uvicorn	ASGI server to run FastAPI	0.30.6
YOLOv8 (Ultralytics)	AI model for person detection, pose estimation, and segmentation	8.2.100
OpenCV	Computer vision library — video capture, image processing, drawing	4.10.0
NumPy	Numerical computation (array operations on images/masks)	(bundled)
Pydantic	Data validation for API request/response models	(bundled with FastAPI)

4.2 Frontend (User Interface)

Technology	Purpose	Version
React	UI component library	19.2.8
Vite	Build tool and dev server (extremely fast)	8.2.2
Tailwind CSS	Utility-first CSS framework	4.3.3
Lucide React	Icon library for clean UI icons	1.42.0
Vanilla JavaScript	WebSocket hook, custom logic	ES2022+

4.3 AI Models Used (Three YOLOv8 Model Files)

Model File	Type	Purpose	Size
`yolov8n-pose.pt`	Pose Estimation	Detects human body keypoints (nose, eyes, ears, shoulders) — used for **seat detection logic**	~6.8 MB
`yolov8n-seg.pt`	Instance Segmentation	Detects person body contours/silhouettes — used for **visual overlays** only	~7.0 MB
`yolov8n.pt`	Object Detection	Standard detection model (available but not actively used in the main loop)	~6.5 MB

> **Key Design Decision:** The project uses **two models simultaneously** — the pose model for accurate seat matching (using head/shoulder keypoints) and the segmentation model purely for drawing beautiful body silhouettes on the video stream.

---

5. System Architecture

```
graph TB
    subgraph Hardware
        CAM["📷 Webcam / Camera"]
    end

    subgraph Backend["Backend Server (Python + FastAPI)"]
        INF["Inference Loop<br/>(YOLOv8 Pose + Seg)"]
        OCC["Occupancy Calculator<br/>(Overlap + Smoothing)"]
        BRD["Broadcast Loop<br/>(WebSocket)"]
        API["REST API<br/>(/seats, /health, /occupancy)"]
        MJPEG["MJPEG Stream<br/>(/video_feed)"]
        STORE["seats.json<br/>(Persistence)"]
    end

    subgraph Frontend["Frontend (React + Vite)"]
        DASH["Dashboard<br/>(Ring Chart + Stats)"]
        VID["Live Video Feed<br/>(MJPEG Stream)"]
        CALIB["Seat Calibrator<br/>(Drag-to-Draw)"]
        WSHOOK["WebSocket Hook<br/>(Auto-reconnect)"]
    end

    CAM -->|"Video Frames"| INF
    INF -->|"Detections"| OCC
    OCC -->|"Seat States"| BRD
    OCC -->|"Annotated Frame"| MJPEG
    BRD -->|"JSON over WS"| WSHOOK
    WSHOOK -->|"Occupancy Data"| DASH
    MJPEG -->|"MJPEG Stream"| VID
    API <-->|"Seat Definitions"| CALIB
    API <-->|"Read/Write"| STORE
```

Architecture Type: **Client-Server with Real-Time Streaming**

Layer	Role
Hardware Layer	Webcam captures video frames
AI Processing Layer	YOLOv8 models run inference on each frame
Application Logic Layer	FastAPI handles occupancy calculation, seat management, and broadcasting
Communication Layer	WebSocket (for real-time data) + MJPEG (for video) + REST API (for CRUD operations)
Presentation Layer	React dashboard displays everything in a polished UI

6. Detailed Data Flow

```
sequenceDiagram
    participant Camera
    participant InferenceLoop
    participant PoseModel as YOLOv8-Pose
    participant SegModel as YOLOv8-Seg
    participant OccCalc as Occupancy Calculator
    participant WebSocket as WebSocket Broadcast
    participant MJPEG as MJPEG Streamer
    participant Browser as User's Browser

    Camera->>InferenceLoop: Capture frame (640×480)

    par Run Both Models in Parallel
        InferenceLoop->>PoseModel: Predict keypoints
        PoseModel-->>InferenceLoop: Keypoints + bounding boxes
    and
        InferenceLoop->>SegModel: Predict segmentation masks
        SegModel-->>InferenceLoop: Body contour polygons
    end

    InferenceLoop->>OccCalc: Torso bboxes + contours

    Note over OccCalc: 1. Head-keypoint verification<br/>2. Build torso bounding box<br/>3.

    OccCalc->>InferenceLoop: Seat occupancy states

    InferenceLoop->>MJPEG: Annotated JPEG frame<br/>(with colored overlays)
    InferenceLoop->>WebSocket: JSON occupancy payload

    par Deliver to Browser
        MJPEG->>Browser: Continuous JPEG stream
    and
        WebSocket->>Browser: JSON data every 1 second
    end

    Note over Browser: Dashboard renders:<br/>• Live video with overlays<br/>• Ring chart +
```

---

7. Backend — Deep Dive

The backend lives in main.py:

7.1 Configuration Constants

- `VIDEO_SOURCE` : default webcam index `0` or test video path.
- `CONFIDENCE_THRESHOLD` : 0.45.
- `SEG_CONFIDENCE_THRESHOLD` : 0.30.
- `NMS_IOU_THRESHOLD` : 0.4.

- `HEAD_KP_CONFIDENCE`: 0.3; `MIN_HEAD_KPS`: 2.
- `OVERLAP_THRESHOLD`: 0.15; `OCCUPANCY_HOLD_SECONDS`: 2.0.

7.2 Data Models

`SeatZoneIn`: normalized rectangle coordinates (`x1`, `y1`, `x2`, `y2`).

`SeatZoneState`: runtime state tracking `occupied`.

7.3 Endpoints

- `GET /health`: Health status.
- `GET /seats`, `POST /seats`, `DELETE /seats/{id}`: REST CRUD operations for seat zones.
- `GET /video_feed`: MJPEG stream.
- `GET /occupancy`: Snapshot of current occupancy.
- `WS /ws`: WebSocket real-time push.

---

8. Frontend — Deep Dive (React + Vite)

8.1 Structure

- `Dashboard.jsx`: Overall layout, ring progress, seat status chips, video feed container.
- `SeatCalibrator.jsx`: HTML Canvas overlay enabling drag-and-draw bounding box calibration directly over the live video.
- `useOccupancySocket.js`: Custom WebSocket hook with auto-reconnection every 3 seconds.
- `vite.config.js`: Proxies `/ws`, `/video_feed`, and `/seats` to port 8000 during local development.

---

9. Key Algorithms & Techniques

1. **Head-Keypoint Verification:** Requires at least 2 visible head keypoints (0-4) with confidence > 0.3 to discard inanimate objects.
2. **Torso Bounding Box:** Bounded exclusively by head and shoulder keypoints (0-6) with 15% padding, excluding limbs.
3. **Best-Match Greedy Assignment:** Each person is mapped exclusively to at most one seat with the highest overlap ratio.
4. **Temporal Smoothing:** Holds an occupied status for 2.0 seconds after detection loss to prevent UI flickering.
5. **Union-Find Contour Merging:** Merges fragmented segmentation contours into a single convex hull.

10. Viva Q&A Highlights (React Version)

- **Q: Why use React + Vite?**

A: React provides reactive state-driven rendering and component modularity. Vite provides lightning-fast Hot Module Replacement (HMR).

- **Q: How does the calibrator work?**

A: An HTML Canvas overlays the live video. Mouse events (`mousedown` , `mousemove` , `mouseup`) capture normalized coordinates that are saved to `seats.json` via `POST /seats` .

- **Q: Why do we need a dev proxy in Vite?**

A: Vite runs on port 5173 while FastAPI runs on 8000. The proxy routes `/ws` and `/video_feed` requests to port 8000 seamlessly, avoiding cross-origin issues during development.